

Oracle AI Database 26ai SQL Cheat Sheet

Oracle AI Database 26ai is a sophisticated database that has features that may not be able to be summarized in a cheat sheet. For complete documentation, see [Oracle Database Documentation](#).

NOTE: You might not have the privilege to execute these commands.

CONNECTING TO ORACLE

Connection options include the following:

- [Command line connections](#)
- [Browser-based connections](#) (Database Actions provided automatically for [Oracle Autonomous Databases](#))
- [VS Code connections](#)

USERS AND SCHEMAS

A schema is a collection of objects (tables, views, etc.). A user is an account with a username and password. Creating a user implicitly creates a schema, and it assigns the user as the owner of that schema. What is commonly called a “database” in other databases is a user/schema pair in Oracle Databases.

```
CREATE USER EXAMPLEUSER
IDENTIFIED BY INIT PASSWORD
DEFAULT TABLESPACE EXAMPLE
QUOTA 10M ON EXAMPLE
TEMPORARY TABLESPACE TEMP
QUOTA 5M ON SYSTEM
PASSWORD EXPIRE;
```

```
DROP USER EXAMPLEUSER;
```

Drops user and all schema objects

```
DROP USER EXAMPLEUSER CASCADE;
```

List all users

```
SELECT USERNAME FROM ALL_USERS;
```

List all users (requires DBA privileges)

```
SELECT USERNAME FROM DBA_USERS;
```

SELECT (QUERY)

Single table SELECT

```
SELECT FIRST_NAME||' '||LAST_NAME FULL_NAME,
       HIRE_DATE
FROM EMPLOYEES
WHERE EMPLOYEE_ID != 100
ORDER BY HIRE_DATE DESC;
```

SELECT with subquery

```
SELECT E.FIRST_NAME||' '||E.LAST_NAME FULL_NAME
FROM EMPLOYEES E
WHERE E.DEPARTMENT_ID = (
    SELECT DEPARTMENT_ID
    FROM DEPARTMENTS
    WHERE DEPARTMENT_NAME = 'FINANCE');
```

Self-join SELECT

```
SELECT A.FIRST_NAME||' '||A.LAST_NAME FULL_NAME,
       B.FIRST_NAME||' '||B.LAST_NAME MGR_NAME,
       A.HIRE_DATE
FROM EMPLOYEES A, EMPLOYEES B
WHERE A.MANAGER_ID = B.EMPLOYEE_ID
ORDER BY MGR_NAME, A.HIRE_DATE DESC;
```

Simple equijoin SELECT

```
SELECT A.FIRST_NAME||' '||A.LAST_NAME FULL_NAME,
       B.JOB_TITLE TITLE,
       B.MAX_SALARY - A.SALARY "DIST TO MAX",
       ROUND(A.SALARY/B.MAX_SALARY,2)*100 "% OF MAX"
FROM EMPLOYEES A, JOBS B
WHERE A.JOB_ID = B.JOB_ID
ORDER BY "% OF MAX" DESC;
```

SELECT with Simple Aggregation

```
SELECT JOB_ID,
       COUNT(JOB_ID) "JOB COUNT"
FROM EMPLOYEES
GROUP BY JOB_ID;
```

SELECT INTO (PL/SQL)

```
DECLARE
    V_EMPLOYEE_NAME      EMPLOYEES.FIRST_NAME%TYPE;
    V_EMPLOYEE_EMAIL     EMPLOYEES.EMAIL%TYPE;
BEGIN
    -- get name and email of employee 100
    -- assign to v_employee_name and v_employee_email
    SELECT FIRST_NAME || ' ' || LAST_NAME,
           EMAIL
    INTO V_EMPLOYEE_NAME,
         V_EMPLOYEE_EMAIL
    FROM EMPLOYEES
    WHERE EMPLOYEE_ID = 100;
    -- write employee email address
    DBMS_OUTPUT.PUT_LINE('Employee 100 is '
        || V_EMPLOYEE_NAME
        || ' and their email is '
        || V_EMPLOYEE_EMAIL);
END;
```

Types of Joins

Equijoins	Band joins
Self join	Cartesian product
Inner join	Outer join
Antijoin	Semijoin

INSERT

```
INSERT INTO DEPARTMENTS (
    DEPARTMENT_ID,
    DEPARTMENT_NAME,
    LOCATION_ID
) VALUES ( 10, 'FINANCE', 100 );
```

You do not need to specify every column in the table with a value. Any unspecified column will be assigned null or its default value. For columns requiring specific values, they must be included.

UPDATE

```
UPDATE EMPLOYEES
SET DEPARTMENT_ID = 30
WHERE FIRST_NAME = 'KIMBERELY';
```

When you run an update, it will update all the rows that match the criteria and return ‘**n rows selected.**’ If no rows are updated, it will return ‘**0 rows selected.**’

DELETE

```
DELETE FROM EMPLOYEES
WHERE FIRST_NAME = 'JANE';
```

When you run a delete, it will delete all the rows that match the criteria and return ‘**n rows deleted.**’ If no rows are deleted, it will return ‘**0 rows deleted.**’

COPYING DATA

```
CREATE TABLE NEW_EMPLOYEES AS
SELECT * FROM HR.EMPLOYEES;
```

```
INSERT INTO NEW_EMPLOYEES
SELECT * FROM HR.EMPLOYEES;
```

There are limitations on using these methods, especially with identity columns, constraints, and similar items. See [CREATE TABLE](#) and [INSERT](#) for more information.

TABLES AND COLUMNS

```
CREATE TABLE DEPARTMENTS (
    ID          NUMBER PRIMARY KEY,
    DEPT_NAME   VARCHAR2(50),
    LOCATION    NUMBER
);
```

Add a unique constraint

```
ALTER TABLE DEPARTMENTS
ADD CONSTRAINT DEPARTMENTS_UNQ
UNIQUE ( DEPT_NAME );
```

Add a not null constraint

```
ALTER TABLE DEPARTMENTS
MODIFY DEPT_NAME CONSTRAINT
DEPT_NAME_NN NOT NULL;
```

Add a check constraint

```
ALTER TABLE DEPARTMENTS
ADD CONSTRAINT CHK_LOCATION CHECK
( LOCATION > 0 );
```

Add a foreign key constraint

```
ALTER TABLE EMPLOYEES
ADD CONSTRAINT FK_DEPT FOREIGN KEY (
    DEPARTMENT_ID )
REFERENCES DEPARTMENTS ( DEPARTMENT_ID );
```

Remove a constraint

```
ALTER TABLE EMPLOYEES
DROP CONSTRAINT FK_DEPT;
```

Add a column

```
ALTER TABLE DEPARTMENTS
ADD COUNTRY_CODE VARCHAR2(2);
```

Rename a column

```
ALTER TABLE DEPARTMENTS
RENAME COLUMN COUNTRY_CODE TO
COUNTRY;
```

Drop a column

```
ALTER TABLE DEPARTMENTS
DROP COLUMN COUNTRY_CODE;
```

Oracle AI Database 26ai SQL Cheat Sheet

OPERATORS

MATH: + (sign), - (sign), + (add), - (subtract), * (multiply), / (divide)

TEXT: || (concatenate)

SETS: union, union all, intersect, intersect all, minus, minus all, except, except all

CONDITIONS

General: =, != (inequality), ^= (inequality), <> (inequality), >, <, >=, <=, true, false, null

Pattern Matching: LIKE

Floating Point: IS [NOT] NAN,
IS [NOT] INFINITE

Logical: NOT, AND, OR

SET: IS [NOT] A SET, IS [NOT] EMPTY,
[NOT] MEMBER OF

JSON: IS [NOT] JSON, JSON VALIDATE,
JSON_EQUAL, JSON_EXISTS,
JSON_TEXTCONTAINS

Membership:
expr1 [NOT] BETWEEN **expr2** AND **expr3**,
EXISTS, [NOT] IN

FUNCTIONS

Numeric Functions

ABS	MOD
ROUND	TRUNC

Text Functions

LOWER	REPLACE
SUBSTR	TRIM
UPPER	INSTR
LENGTH	INITCAP

Datetime Functions

ADD_MONTHS	MONTHS_BETWEEN
SYSDATE	SYSTIMESTAMP

Conversion Functions

TO_CHAR (bfile blob)	TO_CHAR (character)
TO_CHAR (datetime)	TO_CHAR (number)
TO_CLOB (bfile blob)	TO_CLOB (character)
TO_DATE	TO_NUMBER
TO_TIMESTAMP	

NULL-related Functions

NVL	NVL2
-----	------

EXAMPLES (BASIC)

name = 'Bruce'

salary > 15000

commission_pct IS NULL

job_id IN ('SA_MAN', 'SA_REP')

hire_date BETWEEN TO_DATE('01-JAN-18', 'DD-MON-RR') AND SYSDATE

EXAMPLES (COMPOUND)

salary > 5000 AND
department_id IN (20, 70, 80)

commission_pct IS NOT NULL OR
salary > 15000

NOT (salary < 8000)

OPERATORS EXAMPLES

5 + 7 - (-2) * 4 / 3

first_name || ' ' || last_name

```
SELECT FIRST_NAME
FROM EMPLOYEES
UNION
SELECT DEPARTMENT_NAME
FROM DEPARTMENTS;
```

NUMERIC FUNCTION EXAMPLES

```
SELECT ROUND(-123.456,2) ROUNDED,
       TRUNC(-123.456) TRUNCATED,
       ABS(-123.456) ABSOLUTED,
       ABS(MOD(-123,5)) MODDED
FROM DUAL;
```

TEXT FUNCTION EXAMPLES

```
SELECT
  LOWER('HELLO WORLD') LOWTEXT,
  UPPER('hello world') UPTXT,
  INITCAP('HELLO WORLD') INITTEXT,
  TRIM(' Hello World ') TRIMMED,
  LENGTH(' Hello World ') LENEED,
  LENGTH(TRIM(' Hello World ')) LENTRIMED,
  INSTR('Hello World','LO') INSTRED,
  REPLACE('Hello World','H','Y') REPLACED,
  INITCAP(SUBSTR('Hello World',4,8)) SUBBED
FROM DUAL;
```

DATETIME FUNCTION EXAMPLES

```
SELECT
  SYSDATE "Current Date",
  SYSTIMESTAMP "Current Time",
  ADD_MONTHS(SYSDATE, 6) Added,
  MONTHS_BETWEEN(SYSDATE,
  TO_DATE('01-JAN-2023')) Dified
FROM DUAL;
```

CASTING FUNCTION EXAMPLES

```
SELECT
  TO_CHAR(SYSDATE, 'YYYY-MM-DD') formatted,
  TO_DATE('2025-06-11', 'YYYY-MM-DD') convtd,
  TO_NUMBER('12345') numbered
FROM DUAL;
```

JSON FUNCTION EXAMPLES

```
SELECT
  JSON_QUERY('{a:100, b:200, c:300}', '$') JQ,
  JSON_VALUE('{a:100}', '$.a') JV,
  JSON_OBJECT('name' VALUE 'John',
  'age' VALUE 30) JO,
  JSON_ARRAY('apple', 'banana') JA
FROM DUAL;
```

NULL FUNCTION EXAMPLES

```
SELECT last_name,
       NVL(TO_CHAR(commission_pct), 'N/A') commission
FROM employees;
```

```
SELECT last_name, salary,
       NVL2(commission_pct,
       salary+(salary*commission_pct), salary) income
FROM employees;
```

ML FUNCTION EXAMPLES

NOTE: ML examples do not currently run within Live SQL. For the required ML demos, see the [ML GitHub](#) and [ML ESA model](#) repos.

```
-- Predicting candidates for a sale promotion.
SELECT cust_id
FROM mining_data_apply_v
WHERE
  PREDICTION(dt_sh_clas_sample
  USING cust_marital_status,
  education, cust_income_level ) = 1
  AND country_name IN 'Brazil';
```

```
-- Similar texts
SELECT 1-FEATURE_COMPARE(esa_wiki_mod USING
'There are several PGA tour golfers from South Africa' text AND USING 'Retief Goosen won the 2004 U.S. Open' text) similarity FROM DUAL;
```

```
- Dissimilar texts
SELECT 1-FEATURE_COMPARE(esa_wiki_mod USING
'There are several PGA tour golfers from South Africa' text AND USING 'John Elway played quarterback for the Denver Broncos' text) similarity FROM DUAL;
```

VECTOR FUNCTION EXAMPLE

```
WITH vectors AS (
  SELECT
    TO_VECTOR(['1.0, 2.0, 3.0']) AS v1,
    TO_VECTOR(['4.0, 5.0, 6.0']) AS v2
  FROM DUAL
)
SELECT
  -- 1. Converts vector to list of elements
  FROM_VECTOR(v1) AS scalar_values_v1,

  -- 2. Euclidean distance between v1 and v2
  L2_DISTANCE(v1, v2) AS euclidean_dist,

  -- 3. Cosine distance between v1 and v2
  COSINE_DISTANCE(v1, v2) AS cosine_dist,

  -- 4. Magnitude of a vector (like ||v||)
  VECTOR_NORM(v1) AS norm_v1,

  -- 5. Returns # of dimensions in vector
  VECTOR_DIMS(v1) AS dims_v1
```

FROM vectors;